Choupo

The Open Source Chemical Process Simulator

Developer Guide

Version v0.2.0

18 June 2026

Authors: Vítor Geraldes.

Typeset in LaTeX.

Acknowledgment. Choupo was conceived and architected by Vítor Geraldes. Substantial parts of the initial implementation, documentation, and tutorial corpus were produced with assistance from Anthropic Claude Code using Claude Opus/Fable models. This guide is published under human curation, review, and editorial responsibility by Vítor Geraldes.

License

```
    Rights.

  e. Effective Technological Measures means those measures that, in the
     absence of proper authority, may not be circumvented under laws
     fulfilling obligations under Article 11 of the WIPO Copyright
     Treaty adopted on December 20, 1996, and/or similar international
     agreements.

  f. Exceptions and Limitations means fair use, fair dealing, and/or
     any other exception or limitation to Copyright and Similar Rights
     that applies to Your use of the Licensed Material.

  g. License Elements means the license attributes listed in the name
     of a Creative Commons Public License. The License Elements of this
     Public License are Attribution and ShareAlike.

  h. Licensed Material means the artistic or literary work, database,
     or other material to which the Licensor applied this Public
     License.

  i. Licensed Rights means the rights granted to You subject to the
     terms and conditions of this Public License, which are limited to
     all Copyright and Similar Rights that apply to Your use of the
     Licensed Material and that the Licensor has authority to license.

  j. Licensor means the individual(s) or entity(ies) granting rights
     under this Public License.

  k. Share means to provide material to the public by any means or
     process that requires permission under the Licensed Rights, such
     as reproduction, public display, public performance, distribution,
     dissemination, communication, or importation, and to make material
     available to the public including in ways that members of the
     public may access the material from a place and at a time
     individually chosen by them.

  l. Sui Generis Database Rights means rights other than copyright
     resulting from Directive 96/9/EC of the European Parliament and of
     the Council of 11 March 1996 on the legal protection of databases,
     as amended and/or succeeded, as well as other essentially
     equivalent rights anywhere in the world.

  m. You means the individual or entity exercising the Licensed Rights
     under this Public License. Your has a corresponding meaning.


Section 2 -- Scope.

  a. License grant.

       1. Subject to the terms and conditions of this Public License,
          the Licensor hereby grants You a worldwide, royalty-free,
          non-sublicensable, non-exclusive, irrevocable license to
          exercise the Licensed Rights in the Licensed Material to:

            a. reproduce and Share the Licensed Material, in whole or
               in part; and

            b. produce, reproduce, and Share Adapted Material.

       2. Exceptions and Limitations. For the avoidance of doubt, where
          Exceptions and Limitations apply to Your use, this Public
          License does not apply, and You do not need to comply with
          its terms and conditions.

       3. Term. The term of this Public License is specified in Section
          6(a).

       4. Media and formats; technical modifications allowed. The
          Licensor authorizes You to exercise the Licensed Rights in
          all media and formats whether now known or hereafter created,
          and to make technical modifications necessary to do so. The
          Licensor waives and/or agrees not to assert any right or
          authority to forbid You from making technical modifications
          necessary to exercise the Licensed Rights, including
          technical modifications necessary to circumvent Effective
          Technological Measures. For purposes of this Public License,
          simply making modifications authorized by this Section 2(a)
          (4) never produces Adapted Material.

       5. Downstream recipients.

            a. Offer from the Licensor -- Licensed Material. Every
               recipient of the Licensed Material automatically
               receives an offer from the Licensor to exercise the
               Licensed Rights under the terms and conditions of this
               Public License.

            b. Additional offer from the Licensor -- Adapted Material.
               Every recipient of Adapted Material from You
               automatically receives an offer from the Licensor to
               exercise the Licensed Rights in the Adapted Material
               under the conditions of the Adapter's License You apply.

            c. No downstream restrictions. You may not offer or impose
               any additional or different terms or conditions on, or
               apply any Effective Technological Measures to, the
               Licensed Material if doing so restricts exercise of the
               Licensed Rights by any recipient of the Licensed
               Material.
```

```
  a. for the avoidance of doubt, Section 2(a)(1) grants You the right
     to extract, reuse, reproduce, and Share all or a substantial
     portion of the contents of the database;

  b. if You include all or a substantial portion of the database
     contents in a database in which You have Sui Generis Database
     Rights, then the database in which You have Sui Generis Database
     Rights (but not its individual contents) is Adapted Material,
     including for purposes of Section 3(b); and

  c. You must comply with the conditions in Section 3(a) if You Share
     all or a substantial portion of the contents of the database.

For the avoidance of doubt, this Section 4 supplements and does not
replace Your obligations under this Public License where the Licensed
Rights include other Copyright and Similar Rights.


Section 5 -- Disclaimer of Warranties and Limitation of Liability.

  a. UNLESS OTHERWISE SEPARATELY UNDERTAKEN BY THE LICENSOR, TO THE
     EXTENT POSSIBLE, THE LICENSOR OFFERS THE LICENSED MATERIAL AS-IS
     AND AS-AVAILABLE, AND MAKES NO REPRESENTATIONS OR WARRANTIES OF
     ANY KIND CONCERNING THE LICENSED MATERIAL, WHETHER EXPRESS,
     IMPLIED, STATUTORY, OR OTHER. THIS INCLUDES, WITHOUT LIMITATION,
     WARRANTIES OF TITLE, MERCHANTABILITY, FITNESS FOR A PARTICULAR
     PURPOSE, NON-INFRINGEMENT, ABSENCE OF LATENT OR OTHER DEFECTS,
     ACCURACY, OR THE PRESENCE OR ABSENCE OF ERRORS, WHETHER OR NOT
     KNOWN OR DISCOVERABLE. WHERE DISCLAIMERS OF WARRANTIES ARE NOT
     ALLOWED IN FULL OR IN PART, THIS DISCLAIMER MAY NOT APPLY TO YOU.

  b. TO THE EXTENT POSSIBLE, IN NO EVENT WILL THE LICENSOR BE LIABLE
     TO YOU ON ANY LEGAL THEORY (INCLUDING, WITHOUT LIMITATION,
     NEGLIGENCE) OR OTHERWISE FOR ANY DIRECT, SPECIAL, INDIRECT,
     INCIDENTAL, CONSEQUENTIAL, PUNITIVE, EXEMPLARY, OR OTHER LOSSES,
     COSTS, EXPENSES, OR DAMAGES ARISING OUT OF THIS PUBLIC LICENSE OR
     USE OF THE LICENSED MATERIAL, EVEN IF THE LICENSOR HAS BEEN
     ADVISED OF THE POSSIBILITY OF SUCH LOSSES, COSTS, EXPENSES, OR
     DAMAGES. WHERE A LIMITATION OF LIABILITY IS NOT ALLOWED IN FULL OR
     IN PART, THIS LIMITATION MAY NOT APPLY TO YOU.

  c. The disclaimer of warranties and limitation of liability provided
     above shall be interpreted in a manner that, to the extent
     possible, most closely approximates an absolute disclaimer and
     waiver of all liability.


Section 6 -- Term and Termination.

  a. This Public License applies for the term of the Copyright and
     Similar Rights licensed here. However, if You fail to comply with
     this Public License, then Your rights under this Public License
     terminate automatically.

  b. Where Your right to use the Licensed Material has terminated under
     Section 6(a), it reinstates:

       1. automatically as of the date the violation is cured, provided
          it is cured within 30 days of Your discovery of the
          violation; or

       2. upon express reinstatement by the Licensor.

     For the avoidance of doubt, this Section 6(b) does not affect any
     right the Licensor may have to seek remedies for Your violations
     of this Public License.

  c. For the avoidance of doubt, the Licensor may also offer the
     Licensed Material under separate terms or conditions or stop
     distributing the Licensed Material at any time; however, doing so
     will not terminate this Public License.

  d. Sections 1, 5, 6, 7, and 8 survive termination of this Public
     License.


Section 7 -- Other Terms and Conditions.

  a. The Licensor shall not be bound by any additional or different
     terms or conditions communicated by You unless expressly agreed.

  b. Any arrangements, understandings, or agreements regarding the
     Licensed Material not stated herein are separate from and
     independent of the terms and conditions of this Public License.


Section 8 -- Interpretation.

  a. For the avoidance of doubt, this Public License does not, and
     shall not be interpreted to, reduce, limit, restrict, or impose
     conditions on any use of the Licensed Material that could lawfully
     be made without permission under this Public License.

  b. To the extent possible, if any provision of this Public License is
     deemed unenforceable, it shall be automatically reformed to the
     minimum extent necessary to make it enforceable. If the provision
     cannot be reformed, it shall be severed from this Public License
     without affecting the enforceability of the remaining terms and
     conditions.
```

```
  c. No term or condition of this Public License will be waived and no
     failure to comply consented to unless expressly agreed to by the
     Licensor.

  d. Nothing in this Public License constitutes or may be interpreted
     as a limitation upon, or waiver of, any privileges and immunities
     that apply to the Licensor or You, including from the legal
     processes of any jurisdiction or authority.


=======================================================================

Creative Commons is not a party to its public
licenses. Notwithstanding, Creative Commons may elect to apply one of
its public licenses to material it publishes and in those instances
will be considered the "Licensor." The text of the Creative Commons
public licenses is dedicated to the public domain under the CC0 Public
Domain Dedication. Except for the limited purpose of indicating that
material is shared under a Creative Commons public license or as
otherwise permitted by the Creative Commons policies published at
creativecommons.org/policies, Creative Commons does not authorize the
use of the trademark "Creative Commons" or any other trademark or logo
of Creative Commons without its prior written consent including,
without limitation, in connection with any unauthorized modifications
to any of its public licenses or any other arrangements,
understandings, or agreements concerning use of licensed material. For
the avoidance of doubt, this paragraph does not form part of the
public licenses.

Creative Commons may be contacted at creativecommons.org.
```

Trademarks

Choupo is a trademark of TalentGround Lda.

Anthropic, Claude, and Claude Code are trademarks or registered trademarks of Anthropic, PBC.

Linux is a registered trademark of Linus Torvalds.

Python is a trademark or registered trademark of the Python Software Foundation.

Contents

Preface — the contract this code makes with you

If you have written CFD glue for a CFD solver, or stared too long at a black-box solver, you already know why this project exists. Every Newton step is auditable. Every dispatch is explicit. Every dependency is in the C++ standard library. The price of that discipline is that contributors have to play by a few rules. They are short, and they are non-negotiable.

This guide is opinionated on purpose. Read §1 carefully — those three principles override everything else — and treat §23 as the project's only blunt list of don'ts.

1 Philosophy — the contract we promise

What you'll learn. The three invariants that every contribution is judged against, in plain language.

1. **Pedagogical clarity beats elegance.** The simulator must be readable by a chemical-engineering student in their third year. No template metaprogramming, no auto-magic, no clever short-circuits. Every Newton iteration and every K-value must be inspectable in both the source and the run-time output.
2. **Source transparency — no hidden registration.** Every factory base class has an *explicit* `registerBuiltins()` called from `main.cpp`. No `REGISTER_TYPE(MyClass)` macros, no static initialisers, no linker-dependent constructors. The student grep-searches `registerBuiltins` and *sees* every built-in type.
3. **Dependency purity — no copyleft or restrictive libraries linked into the binary.** C++ 17, the C++ standard library, and GNU make. Nothing else. No Boost, no Eigen, no Sundials, no CMake. Hand-rolled Newton, Gauss elimination, RK4, Wegstein, Michelsen.

Key idea. An "industrial-grade" contribution that violates any of these will be rejected even if it converges better and runs faster.

2 Repository tour

> *What you'll learn.* What lives where. Useful as a map; you'll come back to it.

```
Choupo/
|-- Makefile                        top-level: release/debug switch
|-- make/
|   |-- compiler.mk                 CXX, CXXFLAGS, PLATFORM detection
|   |-- rules.mk                    pattern rules + dependency tracking
|   `-- wasm.mk                     Emscripten WASM build (gui/)
|-- build/<PLATFORM>[Debug]/        all.o/.d/binary (gitignored)
|-- choupoSolve                symlink -> last built binary
|-- etc/bashrc                      sourceable shell env
|-- bin/                            runCase, runApplication, listCases, cleanCase
|-- data/
|   `-- standards/                   the FROZEN, committee-managed reference tree
|       |-- components/              <name>.dat --- component catalogue (arity-1)
|       |-- materials/               <name>.dat --- material database
|       |-- membranes/               <name>.dat --- NF/RO membrane catalogue
|       |-- utilities/               <name>.dat --- plant-utility catalogue
|       |-- binaryPairs/             NRTL / Wilson / UNIQUAC pair files (arity-2)
|       |-- henrysLaw/               Henry's-law pairs (arity-2)
|       |-- solution/                <solute>-<solvent>.dat --- aqueous TIER (arity-2)
|       |-- electrolyte/             ions.dat + Pitzer/eNRTL catalogues (arity-2)
|       |-- eos/<model>/             model-keyed binaryInteractions (k_ij)
|       `-- unifac/                  group-interaction tables
|-- docs/                           THIS FILE + userGuide.{md,tex,pdf}
|-- src/
|   |-- core/                        Dictionary, SimulationResult, Constants, Types
|   |-- streams/                     ProcessStream
|   |-- thermo/                      Component, ThermoPackage, Database +
|   |                                 phase/, vaporPressure/, activityCoefficient/,
|   |                                 equationOfState/, heatCapacity/, membrane/
|   |-- materials/                   Material, MaterialRegistry
|   |-- solver/                      NewtonRaphson, NewtonND, Wegstein, StabilityTest
|   |-- unitOperations/              UnitOperation base + per-category folders
|   |-- outerDriver/                 OuterDriver base + SweepDriver + FitBinaryPair
|   |-- postProcessing/              PostProcessor base + Sizing/Costing passes
|   |-- propertyOps/                 PropertyOperation base (choupoProps)
|   |-- control/                     Controller base + PID/Schedule (choupoCtrl)
|   |-- reporting/                   BalanceMath, EnergyBalanceReport (the ledger)
|   |-- io/                          SolutionWriter --- OpenFOAM-style instants
|   `-- applications/                choupoSolve / choupoBatch / choupoCtrl /
|                                      choupoProps -- one main.cpp per binary
|-- tests/                          regression scripts (bin/runTests)
|-- tutorials/                      end-to-end cases (~190, under
|                                      steady/ batch/ ctrl/ props/ plant/)
|-- gui/                            web GUI (see CLAUDE.md Sec. 12)
|-- LICENSE                         GPL-3.0-or-later
`-- README.md
```

Key idea. Data lives under `data/standards/`, write-guarded. The engine *refuses* to write under `data/standards/` — adding a number there is a deliberate curation act, not a side effect of a run. WHERE a number belongs (an intrinsic field in `components/<name>.dat`, a pair file in a catalogue, or an operation's `constant/`) is decided by the **data-governance doctrine** (`docs/ai/data-doctrine.md`): the canonical reference for every curation question, summarised in §19.

Key idea. There is *one binary per problem class*, four in all — they share the same library under `src/{core,thermo,solver,...}` and differ only in their `applications/<name>/main.cpp`:

- `choupoSolve` — steady-state simulation, $F(\mathbf{x}) = 0$ (root-finding);
- `choupoBatch` — batch / recipe-driven, per-vessel ODE integration + time-triggered events;
- `choupoCtrl` — dynamic continuous + control, $d\mathbf{Y}/dt = f(\mathbf{Y}, \mathbf{u}, t)$ with PID/schedule controllers;
- `choupoProps` — property evaluation + parameter fitting (a distinct problem class: neither root-finding nor integration).

We split by *problem class*, never by numerical *strategy* (no `*Foam`-style proliferation): within each binary the strategies (Newton, Wegstein, RK4, Nelder–Mead, ...) are run-time selections via dicts, not compile-time ones.

3 Build system

```
make all              # release, -O2, build/linux64Gcc/
make MODE=debug all   # -O0 -g3, build/linux64GccDebug/
make print            # show resolved variables
make clean            # rm objs + binary for current MODE
make distclean        # rm entire build/
```

Sources are discovered with `find src -name '*.cpp'` — drop a new `.cpp` anywhere under `src/` and it picks up automatically. Header dependencies are auto-tracked via `-MMD -MP` (`.d` files alongside each `.o`).

Platform string is derived from `uname -s -m`. Currently only `linux64Gcc` is exercised; `darwin64Clang` and `linux64Clang` should "just work" with the same Makefile (modulo `<filesystem>` linking on older toolchains — see §15).

The WASM build (`make wasm`) is a sibling target — see `make/wasm.mk` and `CLAUDE.md` §13 for the Emscripten quirks.

4 Build architecture: shared library, optional modules, case code

Since the build is *dynamic* and *modular*. Three forces drove it: incremental compilation (change one binary, don't rebuild the engine), a clean extension story (users add unit ops without touching `src/`), and — load-bearing for the modular architecture — making it hard to produce a standalone binary that carries the whole engine inside. The model: an engine built as a shared library, and small binaries that *reference* it.

> **Key idea.** A binary never embeds the engine; it links `libchoupo.so` at build time and finds it at run time via `rpath`. A binary copied to a machine without the library simply does not start. This is the same model — one engine, many small binaries that share it, so rebuilding a binary never rebuilds the engine.

4.1 The shared engine library (`libchoupo.so`)

`make lib` archives every engine object (everything under `src/` except the per-application entry points) into a single shared object:

```
make lib            # -> build/<plat>/libchoupo.so   ($(CXX) -shared)
```

The engine is compiled `-fPIC` (position-independent) on Linux. Each binary then links `-lchoupo` with `rpath` set to `$ORIGIN` — the binary's own directory — which is where the `.so` is built; the project-root symlinks resolve to the real binary location, so `$ORIGIN` still points at the `.so`. The result is tiny binaries:

Artefact	Size	Contains the engine?
`libchoupo.so`	1.7 MB	it *is* the engine
`choupoSolve` (binary)	86 KB	no — references it
`choupoCase` (a user's)	∼100 KB	no — references it

> **Common pitfall.** The `win64MinGW` cross-build keeps *static* linking on purpose: the generated `.exe` is self-contained. The `LIB_DEP` / `LIB_LINK` Makefile variables carry the platform branch (objects directly for `win64MinGW`, `-lchoupo` on Linux).

4.2 Optional modules — a clean modular split

The engine can carry OPTIONAL modules as separate `.so` files, so a slim core (the teaching engine) builds without them. The membrane module is Vítor's NF/RO research code — conceptually distinct from the core and kept in its own library so a core-only build is possible. Everything is GPL-3.0-or-later; the split is modular, not a licence boundary:

Library	Contents	Licence
`libchoupo.so`	core (thermo, solver, unit ops, reports)	GPL-3.0-or-later
`libchoupoMembrane.so`	NF/RO membrane (`spiralWoundModule`)	GPL-3.0-or-later

The mechanism is the same link-time stub-swap used for user code (§4.3), applied to a module:

1. The optional unit op lives under its own folder (`src/unitOperations/membrane/`); its objects are filtered *out* of `LIB_OBJS` and compiled into `libchoupoMembrane.so` instead.
2. It is **not** registered in `UnitOperation::registerBuiltins()` (the core must not depend on it). Instead a `registerMembrane.cpp` in the module defines a `registerModules()` hook that registers the type via the usual explicit factory.
3. `choupoSolve/main.cpp` calls `registerModules()` right after `registerBuiltins()`. The core binary links an empty `registerModulesStub.cpp` *only* when the module is absent; the Makefile (`MEMBRANE_PRESENT := $(wildcard src/unitOperations/membrane)`) links exactly the `.so` *or* the stub, never both.

```
# slim core-only build: remove the module source, rebuild
rm -rf src/unitOperations/membrane && make all
# -> only libchoupo.so is produced; the membrane type is unknown:
#    "unknown type 'spiralWoundModule'"
```

To add another optional module, do exactly what the membrane does: a folder under `src/`, kept out of `registerBuiltins()`, with its own `registerModules()` compiled into its own `.so`. (When a *second* module appears, generalise the single `registerModules()` hook into a small list — still explicit, still no auto-registration.)

4.3 Case-local code (`case/code/`)

A user — a student, a customer — may write their own C++ under a case's `code/` folder and have it compiled into a per-case binary. Build-time only: **no runtime `dlopen`, no macro self-registration**, faithful to the glass-box credo.

```
case/
  code/
    MyOp.H  MyOp.cpp         # the user's unit op (derives UnitOperation)
    registerUserTypes.cpp    # defines registerUserTypes() -> registerType("myOp",...)
    options                  # OPTIONAL: extra -I / -l / flags (see below)
  system/  constant/
```

What `code/` can register. The primary, tutorialised use is a new *unit operation*. But `registerUserTypes()` runs *after* every `registerBuiltins()`, so it can register **any type with an explicit factory** — the same hook, the same pattern:

What	via	Status
unit operation	`UnitOperation::registerType`	demonstrated (userOp01/02)
thermo model (γ, EoS, P^{sat}, c_p)	`ActivityModel::registerModel`, ...	possible, no tutorial yet
outer driver (sweep / optim / DesignSpec)	`OuterDriver::registerType`	possible
report / sizing / costing	`Report::registerType`, ...	possible

What the user writes. Three files in `code/`: the unit op (`MyOp.{H,cpp}`, a normal class deriving `UnitOperation` with a `solve()` override), and `registerUserTypes.cpp`, which puts the class in the factory:

```
// case/code/registerUserTypes.cpp
void registerUserTypes() {
    Choupo::UnitOperation::registerType(
        "myOp", [] { return std::make_unique<Choupo::MyOp>(); });
}
```

How it is compiled. `bin/buildCode` runs one ordinary `g++` (it prints the exact command — nothing hidden):

```
g++ -std=c++17 -O2 -I src \              # (1) the engine's headers
    build/<plat>/.../choupoSolve/main.o \  # (2) the engine main() (calls
                                           #     registerUserTypes)
    case/code/registerUserTypes.cpp \    # (3) the user's registration
    case/code/MyOp.cpp \                 # (4) the user's unit op
    -Lbuild/<plat> -lchoupo [-lchoupoMembrane] \  # (5) engine.so (+ optional module)
    -Wl,-rpath,build/<plat> \            # (6) where to find the.so at run time
    -o case/code/.build/choupoCase      # (7) the per-case binary (~100 KB)
```

The user's `.cpp` are compiled fresh each time (their `.o` are temporaries in `/tmp`, dropped after linking — only the binary remains). The binary links the engine `.so` DYNAMICALLY (5): it references the engine, never embeds it.

The trick — who supplies `registerUserTypes()`. The engine's `main.o` *calls* `registerUserTypes()` (right after `registerBuiltins()`) but does *not* define it — an unresolved symbol. The linker resolves it from exactly one place, and that place is the build-time choice:

Binary	`registerUserTypes()` from	Result
stock `choupoSolve`	`registerUserTypesStub.cpp` (empty)	registers nothing
the case's `choupoCase`	`case/code/registerUserTypes.cpp`	registers `myOp`

In the `buildCode` command above the stub is *not* linked — only the user's file is — so the linker resolves the symbol with the user's definition. Plain symbol resolution, no weak symbols, no `dlopen`, no macros. (Optional modules use the same pattern with `registerModules()`, §4.2.)

How it is used — at run time.

```
choupoCase runs:
  main()
    -> UnitOperation::registerBuiltins()  // cstr, flash, mixer,...
    -> registerModules()                  // optional modules (membrane.so)
    -> registerUserTypes()   // THE USER'S: registry()["myOp"] = constructor
  Flowsheet reads  type myOp;  in flowsheetDict
    -> UnitOperation::New("myOp")         // look up in the std::map
    -> the user's constructor -> object -> the user's solve()
```

The factory is just a `std::map<string, function>`; `registerType` adds an entry, `New` looks one up. `myOp` sits beside the built-in types — no special case.

What `solve()` receives — streams and thermo. The op's whole world arrives through two arguments:

```
int solve(const DictPtr& dict, const ThermoPackage& thermo, int verbosity);
```

Streams (the `dict`). Choupo is sequential-modular: an op does NOT rummage a global stream registry. The `Flowsheet` *injects* the unit's inlet stream into the `dict` as sub-dicts, plus the unit's own `operation` block:

```
auto feed = dict->subDict("feed");        // F (kmol/s SI), T, P of the inlet
auto comp = dict->subDict("composition"); // mole fractions z_i
auto oper = dict->subDict("operation");   // YOUR parameters (from flowsheetDict)
```

You read your inputs, compute, and publish outputs into `produced_`; the Flowsheet binds them to the next units by the `outputs (... )` order. (For several inlets, the unit declares `inputs (a b )` and reads each.)

Thermodynamics (the `thermo`). The `ThermoPackage&` gives full access to whatever models the case configured — you do not need to know *which* (γ = NRTL? EoS = SRK?); you call the method and the selected model answers:

```
thermo.n();  thermo.indexOf("benzene");  thermo.comp(i);  // count, index, Component
thermo.comp(i).vp().Psat_Pa(T);                            // vapour pressure (Antoine/...)
thermo.Kvec(T, P, x, y);                                   // K-values (the case's gamma-phi)
thermo.activity().gamma(T, x);                             // activity coefficients gamma_i
thermo.Hliquid(T, x);  thermo.Hvapour(T, y);              // sensible-datum enthalpies
thermo.H_real(T, P, y);                                    // real gas (H_ig + EoS departure)
thermo.H_stream_formation(T, P, vf, z);                    // elements-datum (carries Hf)
```

So a user op can flash, balance energy, or solve an equilibrium with the case's thermo — exactly what the built-in CSTR / flash / column do; it is a first-class citizen, not an add-on.

> **Common pitfall.** **Transport properties (viscosity, conductivity, diffusivity) do not exist yet** — they are on the roadmap as a `TransportModel` with a factory. When they land they will be reachable through the same `thermo` handle; today a user op has thermodynamics only.

Command	What it does
`buildCode <case>`	*compiles only* (shows the exact g++ command, reports compiler errors); the development-loop command
`runCase <case>`	*runs* the case — and if it has a `code/` folder, compiles it first (visibly) via `buildCode`

Both step up automatically if invoked from inside the `code/` folder. External dependencies (a property library, GSL, Eigen, ...) go in an optional `code/options` file:

```
# case/code/options
INCLUDE = -I/opt/coolprop/include
LIBS    = -L/opt/coolprop/lib -lCoolProp
FLAGS   = -O3
```

Common pitfall. User code lives in `case/code/`, **never** in `src/`. Three reasons: isolation (the engine tree stays auditable), authorship / licence (`src/` is GPL-3.0-or-later; the user owns their `code/`), and pedagogy (a clear engine-vs-mine boundary). Those external libraries are the *user's* dependencies on the *user's* code — the engine itself stays dependency-free.

5 Core types

> *What you'll learn.* Four types live in `src/core/` and `src/streams/` and propagate through every layer of the simulator: get familiar with them before touching anything else.

5.1 `Dictionary`

hierarchical plain text. Every case dict is parsed into a `Dictionary`. Key operations:

```
auto d = Dictionary::fromFile("system/flowsheetDict");

scalar  T = d->lookupScalar("T");                          // raw
scalar  P = d->lookupScalar("P", Dims::pressure);          // dim-checked
auto    sub = d->subDict("operation");
auto    units = d->lookupDictList("units");
auto    comps = d->lookupWordList("components");

// Path-based mutation --- used by outer drivers
d->setScalarAtPath("units[0].operation.refluxRatio", 3.2);
```

The tokenizer treats `[` and `]` as word chars (so paths work). Optional `FoamFile` headers are skipped silently for compatibility with files from other tools.

Three input forms for scalars. The parser accepts named-unit (`P 1 bar;`), the bracket (`A_w [-1 2 1 0 0] 1.5e-12;`), and raw SI (`R 0.5;`). Named and bracket forms tag the entry with its physical dimensions `[M L T Theta N]` so the dim-checked `lookupScalar(key, expectedDims)` overload can throw a clear error on mismatch. Raw entries pass unchecked — the caller is asserting the value is canonical SI.

Available named dimensions live under `Dims::` in `src/core/Dimensions.H` (`pressure`, `molarFlow`, `massFlow`, `temperature`, `length`, `area`, `volume`, `concentration`, `molarHeatCap`, `viscosity`, `diffusivity`, `permeabilityWater`, `massTransferCoeff`,...). Adopt them at the call sites where the unit op reads its operating parameters.

5.2 `ProcessStream`

What travels between unit operations (units: SI canonical):

```
struct ProcessStream {
    std::string         name;
    scalar              F;       // total FLUID molar flow [kmol/s]
    scalar              T;       // K
    scalar              P;       // Pa
    std::vector<scalar> z;       // FLUID mole fractions, size = NC
    scalar              vf;      // vapour fraction (VLE outputs)
    std::vector<scalar> s;       // solid molar flow per component [kmol/s]
    ParticleSizeDist    psd;     // size distribution of the solid phase
    std::string         category;// "" = process; non-empty = utility leg
};
```

> **Key idea.** `F`, `z` and `vf` describe the *fluid* phases only; the **solid is carried alongside in `s[]`** (kmol/s per component, empty or all-zero = no solids), with its size distribution in `psd`. A crystalliser writes its crystals into `s[]`; the balance machinery (`src/reporting/BalanceMath.H`) reads `s[]` so mass *and* energy close over the solid too (see §20). The `category` tag (populated by `utility <name>;`) lets the energy ledger separate a utility leg — heat crossing the boundary — from a process stream.

Mass-basis flow is a derived view, not a stored field — if you need kg/s, call `F_mass(stream, thermo)` from `src/streams/StreamMass.H`, which computes $F \cdot \sum_i z_i \, \mathrm{MW}_i$ on demand. One source of truth for the flow (molar); mass cannot drift out of sync.

When you produce streams in your unit-op, fill *all* fields. At print sites you typically scale by 3600 for kmol/h and by 10^{-5} for bar, or hand the value plus its dimensions to `DisplayUnits::instance().format(v, dims)` which honours the case's `controlDict.units` preferences.

5.3 `SimulationResult`

What a single pass produces — input to post-processors and outer drivers:

```
struct SimulationResult {
    std::map<std::string, ProcessStream>     streams;
    std::map<std::string, std::map<std::string, scalar>> kpis;
    std::map<std::string, EquipmentSizing>   sizings;      // SizingPass
    std::map<std::string, CostBreakdown>     costs;        // CostingPass
    bool                                     converged;
};
```

The `kpis` map is keyed `unitName` → `{ kpiName` → `value }`. Outer drivers read it; post-processors append to `sizings` / `costs`. (The `sizings` field was named `dimensions` until; renamed because `Dimensions` is now the physical-dimensions tag.)

5.4 `ThermoPackage`

Holds:

- components (vector of `Component`),
- one or more `Phase` instances (vapor / liquid / solid),
- convenience getters: `nComponents()`, `component(i)`, `phase(name)`, `Psat(i, T)`, `gamma(phase, x, T)`, etc.

Every unit operation receives a `const ThermoPackage&` in `solve()`.

Two enthalpy datums — and why only one survives. The package exposes enthalpy on two references:

- `Hliquid(T,x)` / `Hvapour(T,y)` — the **sensible** datum, anchored at an arbitrary T_{ref}. Fine for a single-stream heat-up, useless across a reactor: it silently drops the heat of reaction.
- `H_stream_formation(T,P,vf,z)` — the **elements** datum (formation enthalpies at 298.15 K, the same reference as the ideal-gas H_{ig}). The reaction enthalpy then *emerges* as the difference of formation enthalpies, so a balance closes *through* a reactor, a crystalliser, or a heat of solution.

> **Key idea. One datum, no fallback (§20).** The energy-balance ledger runs on the elements datum *only*. The former silent "fall back to the sensible datum" arm was **deleted**: a component missing its `gibbsFormation` now *throws*, naming the component, so a curation gap is loud, never a quietly wrong closure.

6 The factory pattern — explicit, no macros

> *What you'll learn.* The single registration mechanism used everywhere in the code, and why we refuse the macro-based alternative.

Every extensible base class follows this template:

```
class UnitOperation {
public:
    using Factory = std::function<std::unique_ptr<UnitOperation>()>;

    static void registerType(const std::string& name, Factory f);
    static std::unique_ptr<UnitOperation> New(const std::string& type);
    static std::vector<std::string> availableTypes();
    static void registerBuiltins();          // <-- called in main()

    // ---- virtual interface ----
    virtual const std::string& type() const = 0;
    virtual int  solve(const DictPtr&, const ThermoPackage&, int verbosity) = 0;
    virtual std::vector<ProcessStream> producedStreams() const { return {}; }
    virtual std::map<std::string, scalar> kpis() const { return {}; }

protected:
    // helpers...
};
```

`registerBuiltins()` (in the corresponding `.cpp`) is the single place that *lists every type*:

```
void UnitOperation::registerBuiltins()
{
    auto reg = [](const std::string& n, Factory f) { registerType(n, f); };

    reg("isothermalFlash", []{ return std::make_unique<IsothermalFlash>(); });
    reg("flash",           []{ return std::make_unique<IsothermalFlash>(); });
    reg("adiabaticFlash",  []{ return std::make_unique<AdiabaticFlash>();  });
    reg("bubbleT",         []{ return std::make_unique<BubblePoint>();     });
    reg("dewT",            []{ return std::make_unique<DewPoint>();        });
    reg("cstr",            []{ return std::make_unique<CSTR>();            });
    reg("pfr",             []{ return std::make_unique<PFR>();             });
    reg("heatExchanger",   []{ return std::make_unique<HeatExchanger>();   });
    reg("HX",              []{ return std::make_unique<HeatExchanger>();   });
    reg("mixer",           []{ return std::make_unique<Mixer>();           });
    reg("splitter",        []{ return std::make_unique<Splitter>();        });
    reg("distillationColumn", []{ return std::make_unique<DistillationColumn>(); });
    reg("column",             []{ return std::make_unique<DistillationColumn>(); });
}
```

`main.cpp` calls each `registerBuiltins()` once at startup, in order. That is the *entire* registration story. No magic, no surprises.

> **Common pitfall.** Do *not* introduce `REGISTER_TYPE(...)` macros, anonymous-namespace static instances, or any pattern that runs registration as a side effect of TU loading. We have rejected this approach explicitly because the linker can drop unused TUs and the failure mode is silent.

7 Adding a new unit operation

> *What you'll learn.* The five moving parts in outline — skeleton, implementation, registration, tutorial, regression. The full *tim-tim-por-tim-tim* walkthrough, with a real worked example and the mandatory WASM rebuild, is in §21.1; this is the at-a-glance version.

7.1 Step 1 — Skeleton

Create `src/unitOperations/<category>/MyOp.H`:

```
#pragma once
#include "core/Dictionary.H"
#include "thermo/ThermoPackage.H"
#include "unitOperations/UnitOperation.H"

namespace Choupo {

class MyOp : public UnitOperation {
public:
    MyOp() = default;
    const std::string& type() const override { return type_; }

    int solve(const DictPtr&         dict,
              const ThermoPackage&   thermo,
              int                    verbosity) override;

    std::vector<ProcessStream> producedStreams() const override { return produced_; }
    std::map<std::string, scalar> kpis() const override { return kpis_; }

private:
    inline static const std::string type_ = "myOp";
    std::vector<ProcessStream>      produced_;
    std::map<std::string, scalar>   kpis_;
};

} // namespace Choupo
```

7.2 Step 2 — Implementation

`src/unitOperations/<category>/MyOp.cpp` — implement `solve()`. Boilerplate to follow:

```
int MyOp::solve(const DictPtr& dict, const ThermoPackage& thermo, int verbosity)
{
    // 1. The Flowsheet INJECTS the inlet + your params as sub-dicts of ‘dict‘
    //    (sequential-modular: no global stream registry to rummage).
    auto feed = dict->subDict("feed");          // F (kmol/s SI), Tfeed, Pfeed
    auto comp = dict->subDict("composition");   // mole fractions z_i
    auto op   = dict->subDict("operation");     // YOUR parameters
    const scalar F     = feed->lookupScalar("F",     Dims::molarFlow);
    const scalar Tfeed = feed->lookupScalar("Tfeed", Dims::temperature);
    const scalar T     = op->lookupScalar("T",       Dims::temperature);

    // 2. Echo inputs (verbosity >= 3 --- the pedagogical default)
    if (verbosity >= 3)
        std::cout << "  [MyOp] T = " << T << " K, F_in = "
                  << (F * 3600.0) << " kmol/h\n";   // print in kmol/h, store SI

    // 3. Solve the model --- use solver/ classes for Newton/Wegstein.
    //    newton1D(f, dfdx, x0, NROptions) -> NRResult { root, iterations, converged }
    solver::NROptions nro;  nro.lower = 200.0;  nro.upper = 700.0;
    auto r = solver::newton1D([&](scalar x){ /* g(x) */ return 0.0; },
                              [&](scalar x){ /* g’(x) */ return 1.0; },
```

```
                                /*x0=*/Tfeed, nro);

    // 4. Build produced streams into produced_ (order = your contract;
    //    the Flowsheet binds them to names via 'outputs (... )')
    produced_.clear();
    ProcessStream out;
    out.name = "myOp_out";
    out.F = F;  out.T = T;  out.P = feed->lookupScalar("Pfeed", Dims::pressure);
    out.z.resize(thermo.n());                    // fill ALL fields
    produced_.push_back(out);

    // 5. Populate KPIs (they feed sizing / costing / sensitivity)
    kpis_.clear();
    kpis_["T_out"] = T;
    kpis_["F"]     = F;

    return 0;     // 0 == converged
}
```

7.3 Step 3 — Register

Edit `src/unitOperations/UnitOperation.cpp`:

```
#include "<category>/MyOp.H"
//...
void UnitOperation::registerBuiltins() {
    //...
    reg("myOp", []{ return std::make_unique<MyOp>(); });
}
```

7.4 Step 4 — Tutorial

Create `tutorials/myOp01_quick_smoke/` with `system/controlDict`, `system/flowsheetDict`, and `constant/thermoPackage`. Keep the case trivial so it doubles as a regression test.

7.5 Step 5 — Build, WASM, regression

```
make all                 # recursive find picks the new .cpp up automatically
make wasm-gui            # MANDATORY: the GUI's WASM is a SEPARATE build (see below)
bin/runTests             # NaN/inf guard on every case + golden-master KPIs
```

Common pitfall. **Rebuild the WASM or the browser will not see your type.** The WASM solver is a separate compile from the native binary. Symptom of a stale `.wasm`: the native run works, but the GUI errors `UnitOperation::New: unknown type 'myOp'`. Fix with `make wasm-gui` (force a clean one with `make wasm-clean && make wasm` if it says "Nothing to be done"). Never validate with a bare `choupoSolve > /dev/null && PASS` loop — a solver can return NaN with exit code 0; `bin/runTests` is the only honest gate (§14).

All previously-passing tutorials *must* still pass.

8 Adding a new thermodynamic model

Same explicit-factory spirit as everywhere else, with `ActivityModel` / `EquationOfState` / `VaporPressureModel` / `HeatCapacityModel` / `Phase` as the base. The one difference from `UnitOperation` is **the factory takes construction arguments** (a thermo model needs the configuring sub-dict, and usually the component list), so its `Factory` signature is not the nullary unit-op one — match the base you are extending:

Base	`Factory` / register call
`VaporPressureModel`	`(const DictPtr&)`; `registerModel(name, f)`
`HeatCapacityModel`	`(const DictPtr&)`; `registerModel(name, f)`
`ActivityModel`	`(const DictPtr&, const std::vector<std::string>&);` `registerModel(name, f)`
`EquationOfState`	`(const DictPtr&, const std::vector<Component>&); registerModel(name, f)`

The detailed, copy-pasteable recipe — with the validation-first AAD step and the EoS data-doctrine path — is in §21.2. The short version, adding UNIQUAC as an activity model:

1. `src/thermo/activityCoefficient/UNIQUAC.{H,cpp}` inheriting `ActivityModel`. Override the pure virtuals `gamma(scalar T_K, const sVector& x)`, `n()`, `modelName()`; a constructor `UNIQUAC(const DictPtr&, const std::vector<std::string>&)` reads the pair parameters.
2. In `src/thermo/activityCoefficient/ActivityModel.cpp`, add the `#include` and *one* line in `registerBuiltins()`:

```
#include "UNIQUAC.H"
//...
void ActivityModel::registerBuiltins() {
    registerModel("ideal",  [](const DictPtr&, const std::vector<std::string>& n)
        -> std::unique_ptr<ActivityModel>
        { return std::make_unique<IdealSolution>(n.size()); });
    registerModel("NRTL",   [](const DictPtr& d, const std::vector<std::string>& n)
        -> std::unique_ptr<ActivityModel>
        { return std::make_unique<NRTL>(d, n); });
    registerModel("Wilson", [](const DictPtr& d, const std::vector<std::string>& n)
        -> std::unique_ptr<ActivityModel>
        { return std::make_unique<Wilson>(d, n); });
    registerModel("UNIQUAC",[](const DictPtr& d, const std::vector<std::string>& n)   // new
        -> std::unique_ptr<ActivityModel>
        { return std::make_unique<UNIQUAC>(d, n); });
}
```

3. Use it in a tutorial's `thermoPackage`:

```
activityModel { model UNIQUAC; pairs (... ); }
```

9 Property operations (choupoProps)

choupoProps runs *property operations* rather than a flowsheet. Each inherits PropertyOperation (same explicit factory as everywhere else; registerBuiltins() is called from src/propertyOps/PropertyOperation.cpp). An op reads a state (T, P, composition), a properties list and an output block, and writes a CSV the GUI auto-plots. Built-ins: propertyPoint, propertyScan1D, propertyScan2D, propertyScanTernary, fitParameters, vleConsistency, estimateComponent. Property keywords live in PropertyEvaluator.cpp: per-component Psat_<c>, gamma_<c>, y_eq_<c>, Cp_liquid_<c> (each guarded — e.g. Psat on a component with no vapour-pressure model throws a clear error instead of dereferencing null), and mixture scalars Z, v_molar, H_real, S_real, transport, etc. PropertyScan1D wraps each evaluation in try/catch, writing nan on failure so one undefined component never aborts the whole scan.

propertyScanTernary — ternary phase diagrams by reuse. Walks the composition simplex ($x_1 + x_2 + x_3 = 1$) at fixed (T, P) and, at every node, calls the SAME solver the isothermalFlash unit op calls — so the diagram cannot disagree with a single flash. Three modes:

- bubbleT — a boiling-temperature *surface*: per node BubblePoint::compute; needs only the three vapour pressures (ideal binaries are fine), no liquid-liquid data.
- lle (default) — a *solubility* map: the liquid–liquid flash (PhaseSet::LL) gives a clean binodal + tie-lines, no spurious subcooled vapour (the extraction/decanter view).
- vlle — the full vapour + two-liquid classification (PhaseSet::VLLE).

Each simplex node is independent, so the scan is embarrassingly parallel: the optional shard { k; n; } block computes only rows with $i \bmod n = k$ (round-robin, balanced). The GUI fans $N = \text{cores} - 2$ shards across N WASM workers and merges the partial CSVs (the union of shards reproduces the full grid exactly). No physics is reimplemented — the op is a triangular loop + a classifier + a CSV writer over solveCore/BubblePoint.

Adding a property op. Mirror the recipe of §6: implement src/propertyOps/MyOp.{H,cpp} inheriting PropertyOperation (override type(), run(dict,thermo,verbosity), optionally diagnostics()), then one reg("myOp", ...) line in PropertyOperation::registerBuiltins(). Rebuild the native binary AND the WASM (make wasm-gui) or the browser will not see the new op.

10 Adding components, materials, reactions

These are *data-driven* — no C++ changes needed. WHERE each number belongs is not a matter of taste: it is decided by the data-governance doctrine (`docs/ai/data-doctrine.md`, summarised in §19). Read that before adding or moving a value.

10.1 Components

Choupo's component data cascades through tiers, lowest precedence to highest: `proposed < standard < case < sector < unit`.

- `data/standards/components/<name>.dat` — the **curated central catalogue** shipped with the simulator. Carries only **arity-1 intrinsic** data: identity, `critical`, `gasIdeal`, `liquidPure`, `solid`, `transport`, the `gibbsFormation` datum, and any `eosParameters{}` model-keyed block. Treat it as read-only — the engine *refuses* to write here; a change is a reviewed curation act. A **pair** number (NRTL τ, Henry, k_{ij}, a heat of solution) is **never** copied into the `.dat`; it lives in a catalogue, referenced by name.
- `<case>/constant/components/<name>.dat` (and the same at `<sector>/` and `<unit>/`) — **per-scope overlays** for sample-specific refinements (a particular powder's sorption isotherm, a laboratory measurement).

> **Key idea. The overlay merges block-by-block, NOT field-by-field.** An overlay carrying `solid { rho_p 1610; }` replaces the *entire* `solid{}` block — the standard's `k_v` is *lost*, not deep-merged. A reference-state block is the atomic unit of physical meaning; mixing a sample's `rho_p` with the catalogue's `k_v` would be a silent hidden hybrid. **The curator must copy the whole block they refine.** (This corrects the pre-doctrine "field-by-field" behaviour.)

The file format is identical in both tiers:

```
component <name>;
mw      78.11;           // g/mol
Tc     562.05;           // K
Pc      48.95;           // bar
omega  0.2103;
Tb     353.24;           // K
Hvap_Tb  30720.0;        // J/mol at Tb
Vliq   89.4e-6;          // m^3/mol (Wilson)

vaporPressure
{
    model    Antoine;
    A        4.01814;
    B        1203.835;
    C       -53.226;
}

idealGasHeatCapacity
{
    model       polynomial;
    coeffs      ( -31.368  0.4746  -3.107e-4  8.524e-8 );
    Tmin        200.0;
    Tmax        1500.0;
}

liquidHeatCapacity
{
    model       polynomial;
    coeffs      ( 162.94  -0.3493  9.51e-4  0.0 );
    Tmin        278.0;
    Tmax        500.0;
}

gibbsFormation
{
    dHf_298     82930.0;        // J/mol  (NIST ideal-gas value)
```

```
    s_298         269.20;        // J/(mol*K)  (third-law absolute)
    phase       gas;             // phase in which dHf_298 / s_298 are tabulated
}
```

Source the values from DIPPR / Reid-Prausnitz-Poling and *cite the source in a top-of-file comment* so they stay auditable.

The `phase` field (mandatory in `gibbsFormation`). The reference state for formation enthalpy is documented in the Theory Guide (chapter on the elements datum). The `phase` keyword tells the solver in *which phase* `dHf_298` is tabulated, so `Component::h_formation(T, targetPhase)` can build the right integration path through Hvap / Hfus jumps and the matching c_p leg. Three values are allowed:

phase	required data	typical species
`gas`	`idealGasHeatCapacity`, plus `HvapTb`+Tc+Tb if the liquid leg of any stream is needed	*the convention — all 45 standard volatiles + radicals; follows NIST / JANAF*
`liquid`	`liquidHeatCapacity`; `HvapTb`+Tc+Tb if any stream goes through the gas leg	reserved for compounds whose Hf is published in the condensed datum (rare)
`solid`	`liquidHeatCapacity` (used as the dissolved-solute approximation $c_p^{\text{solid}} \approx c_p^{\text{liq}}$)	crystalline non-volatiles whose Hf cannot be referenced to gas because the compound never vaporises (sucrose)

The default is `gas` for backward compatibility with the earliest component files, but **new components should always declare it explicitly** so the datum is unambiguous at the source of truth. A `solid` or `liquid` declaration without the matching c_p model throws a clear error from `Component::h_formation` the first time a stream containing the component is evaluated.

> **Common pitfall. A missing `gibbsFormation` is now fatal, not papered over.** The energy ledger runs on the elements datum *only* (§20); the old "fall back to the sensible datum with a note" path was deleted. A component that appears in a stream the balance touches but carries no formation datum throws, naming the component — a loud curation gap, never a quietly wrong closure. Curate the eleven historical gaps (or the new component's datum) before relying on the balance.

Add the new component name to the case's `thermoPackage.components (...)` list (or to a per-case `constant/components/<name>.da` overlay if the data is sample-specific) and the new entry is picked up automatically — no recompile needed.

10.2 Materials

`data/materials/<name>.dat`:

```
material <name>;
rho          7900.0;     // kg/m^3
F_M          1.0;        // Guthrie material factor
sigma_y      138.0e6;    // Pa
maxT         700.0;      // K
maxP         50.0;       // bar
```

`MaterialRegistry::loadFrom()` slurps the directory on simulator start. No registration step needed.

10.3 Utilities

`data/standards/utilities/<name>.dat` catalogues a plant utility (steam header, cooling-water loop, hot-oil loop, refrigeration brine). The same registry pattern as materials and membranes: the `UtilityCatalogue::loadFrom(dataRoot)` call in each main slurps the directory at startup; the stream parser then resolves `utility <name>;` keys against it.

```
utility     steamLP;
tier        heating;            // heating | cooling
mechanism   condensation;       // condensation | sensible | evaporation

components  ( water );
state       saturatedVapour;
P           2.5 bar;
T_in        400.3 K;
T_out       400.3 K;             // condensate same T as steam

dutyPerKg   2186000.0;           // J/kg delivered  (Hvap_water at T_sat)
cost        12.0;                // EUR/GJ delivered
costYear    2022;
description "Low-pressure saturated steam header";
```

To add a new utility, drop a new `.dat` into the directory — no C++ change. When the utility loop carries a non-water fluid (oils, salts, brines), the `components` list points at a corresponding entry under `data/standards/components/`; the catalogue ships `dowthermA`, `hitecSalt` and `propyleneGlycol30` pseudo-components for that purpose. See the user guide for the case-author syntax `utility <name>;` and the `utilities` report that aggregates kg/h, MW, and EUR/h per category.

10.4 Reactions

In a case's `constant/reactions`:

```
reactions
{
    ethanol_to_water_test
    {
        stoichiometry
        {
            ethanol      -1.0;
            water        +1.0;
        }
        kinetics
        {
```

```
            type        powerLaw;
            A           1.0e6;        // s^-1
            E           80000.0;      // J/mol
            orders      { ethanol 1.0; }
        }
    }
}
```

Referenced from a CSTR / PFR by name (`reaction ethanol_to_water_test;`).

> **Key idea. The reactions library cascades UP, like `thermoPackage`.** When a branch of a fractal plant (§17) is run *standalone*, its named-reaction library resolves by walking up the parent-folder chain (capped at six levels) exactly as the thermo package does, and **announces its origin** — `reactions library: loaded -- LOCAL` vs `inherited from <parent>/...`. Before this symmetry was restored, a sub-sector run alone died with "reaction not in constant/reactions" because only the local folder was consulted: the two cascades were asymmetric. No-silent-crutch: the cascade is loud.

11 Adding an outer driver / post-processor

11.1 Outer driver

src/outerDriver/MyDriver.{H,cpp} inherits OuterDriver. Override:

```
class MyDriver : public OuterDriver {
public:
    void readFromDict(const DictPtr&) override;
    int  run() override;       // calls simulator_(flowsheetDict_) N times
};
```

The simulator functor and dict templates are injected by main.cpp:

```
auto driver = OuterDriver::New(outerDict);
driver->setSimulator     (simulate);          // SimulationResult(DictPtr)
driver->setFlowsheetDict (flowsheetDict);     // mutable flowsheet template
driver->setThermoDict    (thermoDict);        // mutable thermoPackage
driver->run();
```

The simulator closure captures the thermo dict by shared_ptr, so any in-place setScalarAtPath() the driver makes on thermoDict_ is picked up on the next simulator_() call.

Inside run(), you typically:

1. Mutate flowsheetDict_->setScalarAtPath(...) to change a sweep variable, OR thermoDict_->setScalarAtPath(...) to change a model parameter (e.g. NRTL a_ij).
2. Call simulator_(flowsheetDict_) → get a SimulationResult.
3. Extract KPIs / stream fields, accumulate residuals / responses.
4. Decide next step (LM / Nelder-Mead / GA / . . .).
5. Eventually report and return 0 (converged) or 1 (failed).

Two concrete examples to read first:

- src/outerDriver/SweepDriver.{H,cpp} (∼ 150 LOC) — 1-D sensitivity sweep.
- src/outerDriver/FitBinaryPair.{H,cpp} (∼ 300 LOC) — Marquardt LM with FD Jacobian, mutates both flowsheet (composition) and thermo (pair params).

Register your driver in OuterDriver::registerBuiltins().

11.2 Post-processor

src/postProcessing/MyPass.{H,cpp} inherits PostProcessor. Single method:

```
int run(SimulationResult&) override;
```

Append to result.sizings / result.costs, write CSVs, print to stdout. Register in PostProcessor::registerBuiltins().

PostProcessor::buildChain(postDict) walks postDict top-level keys and instantiates one pass per recognised key. Add yours to the chain-builder in PostProcessor.cpp.

12 Adding a sizing model

EquipmentSize is a sub-factory used by SizingPass. Add src/postProcessing/sizing/MyEquip.{H,cpp} inheriting EquipmentSize:

```
class MyEquip : public EquipmentSize {
public:
    EquipmentSizing size(const std::string& unitName,
                         const SimulationResult& result,
                         const Material&  material,
                         const DictPtr&   designRules) override;
};
```

Use result.streams.at(...) and result.kpis.at(unitName).at(...) to get the process data, plus designRules for the design specs (pressure design, F_M, joint efficiency, ...). Fill EquipmentSizing:

```
EquipmentSizing d;
d.unitName       = unitName;
d.equipmentType  = "myEquip";
d.material       = material.name;
d.values["V_R"]    =...;       // canonical size
d.values["t_wall"] =...;       // m
d.values["weight"] =...;       // kg
return d;
```

Register in EquipmentSize::registerBuiltins().

The Guthrie costing model reads d.equipmentType to dispatch to the right correlation — so if your new equipment needs new cost coefficients, edit src/postProcessing/costing/Guthrie.cpp accordingly.

13 Coding conventions

13.1 Style

- C++ 17. `std::variant`, `std::optional`, `std::unique_ptr`, `std::filesystem` — yes. No raw `new` / `delete`. No `goto`.
- **Brace style:** Allman / open-brace on a new line for function definitions; Egyptian (`if (x) {`) for control flow.
- **Indent:** 4 spaces. No tabs anywhere.
- **Naming:** `snake_case` for locals, `camelCase` for methods, `PascalCase` for types, `UPPER_SNAKE` only for macros (we use exactly one: include guards via `#pragma once`).
- Compile cleanly with `-Wall -Wextra -Wpedantic` and fix anything that fires.

13.2 Includes

- Headers compile in isolation — every `.H` includes what it needs.
- Forward-declare aggressively in headers; pull full definitions only in `.cpp`.
- The build does not use precompiled headers.

13.3 Errors

- User-facing: throw `std::runtime_error` with a message a student can understand. Don't `assert` on user-data invariants.
- Programmer-facing (impossible state): `assert(...)` is fine.
- Never silently swallow errors. If a sub-step fails, surface it.

13.4 Comments

- Default to writing *no* comments. Identifiers should be obvious.
- Equations and references *deserve* a comment. Cite the source (Reid-Prausnitz-Poling page, DIPPR ID, Henley-Seader chapter).
- Top-of-file banner is preserved on every source — do not delete.

13.5 Header banner

Every `.H` and `.cpp` starts with:

```
/*---------------------------------------------------------------------------*\
|  Choupo                                                                     |
|                                                                             |
|  Open-source chemical process simulator                                     |
|                                                                             |
|  Copyright (C) 2026 Vítor Geraldes                                          |
|  SPDX-License-Identifier: GPL-3.0-or-later |
-------------------------------------------------------------------------------
Class
    Choupo::<ClassName>

Description
    <one-paragraph description>
\*---------------------------------------------------------------------------*/
```

Don't omit this on new files — it doubles as licence notice.

14 Regression testing

Regression is run with `bin/runTests`. Run it before every change set; all previously-passing tutorials must still pass.

```
bin/runTests                  # every tutorial (4 binaries + buildCode cases)
bin/runTests <case>           # one case
bin/runTests --record <case> # refresh that case's golden-master 'expected'
```

`runTests` does two checks, both stronger than a plain exit-code loop:

1. **NaN / inf guard — every case.** The structured result block is scanned for NaN or inf values; a hit FAILS the case. This is the check a bare `binary > /dev/null && PASS` loop *cannot* do: a solver can return NaN with exit code 0, and the old loop reported it green. That exact trap once hid a broken `membrane01` (NaN KPIs, exit 0) after a pressure-unit refactor.
2. **Reference KPIs — cases with an `expected` file.** Each line pins a KPI or stream value to its golden-master value with a relative tolerance; any deviation FAILS, printing `got X expected Y`.

> **Key idea.** The `expected` file lives in the case directory. Format (`#` comments allowed):
>
> ```
> # kind   name      key        expected        reltol
> kpi      flash01   V_over_F   0.30398357314   1e-6
> stream   vapor     F          0.00844398814   1e-6
> ```
>
> `kind` is `kpi` (inside the named unit's KPIs) or `stream` (inside the named stream). Pin the numbers worth guarding; `-record` writes them from the current run once you trust it. A case with no `expected` still gets the NaN guard.

At, four canonical cases ship an `expected` (flash01, cstr01, column01, membrane01); the rest run under the smoke + NaN guard. Add an `expected` whenever a case's numbers are load-bearing.

15 Debugging

15.1 Debug build

```
make MODE=debug all
gdb build/linux64GccDebug/choupoSolve
(gdb) r tutorials/flash01_benzene_toluene
```

The debug build leaves a separate symlink `choupoSolve.debug` so the release binary stays usable in parallel.

15.2 Verbosity

`controlDict { verbosity 4; }` enables debug-level prints — every Newton iteration, every K-value evaluation, every Wegstein step. For LL flashes that misconverge, watch for stagnation: $g(x)$ should shrink monotonically near the root.

15.3 Sanitizers

Edit `make/compiler.mk` for the debug branch:

```
ifeq ($(MODE),debug)
    CXXFLAGS += -fsanitize=address,undefined -fno-omit-frame-pointer
    LDFLAGS  += -fsanitize=address,undefined
endif
```

Then rebuild. Useful when a unit-op produces NaN out of nowhere or when adding a new factory and a dispatch crashes.

15.4 The most common failure: NaN propagation

Symptoms: `verbosity 3` shows clean inputs, then `V/F = nan` mid-flash. Almost always a $\log P_i^{\mathrm{sat}}$ with $P_i \leq 0$ (Antoine outside its valid range) or an empty composition (`F = 0`). Add asserts where you suspect it.

16 Architecture deep-dive — the three layers

> *What you'll learn.* The contract between the outer driver, the simulator core, and the post-processor. Read this twice before adding cross-layer logic.

```
+------------------------------------------------------------------+
| Layer 1: OuterDriver  (src/outerDriver/)                         |
|   Sensitivity sweep, optimisation, parameter estimation, MC.     |
|   Owns: simulator_ functor, flowsheetDict_ (mutable).            |
|   Mutates flowsheetDict_ between calls via setScalarAtPath().    |
+------------------------------------------------------------------+
                              |  calls
                              v
+------------------------------------------------------------------+
| Layer 2: Simulator core                                          |
|   runSimulation(flowsheetDict,...)                              |
|     -> ThermoPackage::readFromDict(thermoDict, db)               |
|     -> Flowsheet::solve(flowsheetDict, thermo, verbosity)        |
|     -> SimulationResult { streams, kpis, converged }             |
|   Pure function: same inputs -> same outputs.                    |
+------------------------------------------------------------------+
                              |  result
                              v
+------------------------------------------------------------------+
| Layer 3: PostProcessor (src/postProcessing/)                     |
|   Sizing  -> equipment dimensions (V_R, A, t_wall, weight,...) |
|   Costing -> Guthrie/Turton with CEPCI, USD->EUR                 |
|   (future) Reporting -> PDF / HTML / CSV bundles                 |
|   Augments SimulationResult; never re-enters the simulator.     |
+------------------------------------------------------------------+
```

The contract:

- Layer 1 *never touches the simulator's internal state* — it only mutates the dict it owns and calls the functor.
- Layer 2 *is pure* — `runSimulation(d) == runSimulation(d)` for the same dict. This is what lets Layer 1 do finite-difference Jacobians and Levenberg-Marquardt cleanly.
- Layer 3 *never re-runs the simulator* — it only reads the result. If you need to re-run (e.g. for utility costing), do it via an outer driver.

> **Key idea.** Rule of thumb: if a quantity can be computed from a converged stream table plus KPIs, it belongs in a post-processor. If it requires re-solving the flowsheet, it belongs in an outer driver.

17 The fractal flowsheet — nesting and `flattenNode`

> *What you'll learn.* How a plant of sectors of units collapses to ONE flat solver problem, why the hierarchy is namespace-only, and how a branch runs standalone.

A Choupo case is a **fractal**: a composite *sector* nests leaf units (or further sectors), to any depth. But the solver never sees the tree. `flattenNode` walks the declared children and collapses the hierarchy into **one flat solver problem** with dotted `plant.sector.unit` names — the hierarchy is *authoring and namespace* only, never a recursive solver-within-a-solver. One flat Newton-on-tears recycle solves the whole plant.

> **Key idea. The hierarchy is discovered from the *dicts*, not the filesystem.** `flattenNode` iterates the `children(... )` declared in `flowsheetDict`; a child loads as a sector only when its own `flowsheetDict` exists. This is why the numeric instant directories (§18) sitting beside the sector folders are never mistaken for sectors — they are not declared children and have no `flowsheetDict`.

Lean fractal folders + the dual-reader. A node's topology may live either way, both first-class:

- the classic `<node>/system/flowsheetDict`, or
- the lean `<node>/flowsheetDict` directly at the node root.

A **dual-reader** keeps both loading: the child loader reads `<child>/flowsheetDict` first and *falls back* to `<child>/system/flowsheetDict`; the top-level `choupoSolve` does the same at the case root. Existing fractal cases (esterification, twoSectorDemo) load byte-identically through the fallback — **no forced mass-migration**.

A branch runs alone. Because both `thermoPackage` and the reactions library cascade UP the parent chain (§10), pointing `choupoSolve` at a sub-sector folder runs that branch as a complete case, inheriting the parent's thermo and reactions and announcing what it inherited.

The model-boundary rule. A per-unit `thermo {}` override REPLACES the global models for that unit (components stay global, via `thermoFor`). A stream crossing such a boundary is re-interpreted there. The governing law: H **is the conserved truth,** T **is the model-dependent readout**. A model boundary is not a physical device, so there is no real ΔT to absorb; H-continuity is a convention, not a law. The default is **hold-T, let-H-jump** (the discontinuity is visible in the printed enthalpy); a silent "hold H, flex T" is rejected. The honest opt-in is a **model-boundary audit** — print $\Delta H = H_{\text{down}} - H_{\text{up}}$ at fixed (T, P, z), fold it into the first-law ledger, and hard-refuse across any phase / `vf` / speciation flip. Full rationale: `docs/ai/energy.md`.

18 Solution instants — OpenFOAM-style per-iteration snapshots

> *What you'll learn.* Where the converged (and intermediate) stream tables are written, and why the layout is faithful to OpenFOAM's time directories.

A run writes **durable per-iteration snapshots** of every stream, modelled on OpenFOAM's time directories. `src/io/SolutionWriter` owns the writes; the durability core is atomic (`.tmp` → `rename` + `fsync`, gated on `flushEach`), a restart re-seeds the recycle tear from the latest instant, and a litter sweep purges stale numeric dirs.

The layout — instants at the CASE ROOT. An instant is a bare iteration-*number* directory at the case root, beside `constant/`, `system/` and the fractal sector folders. "The time speaks for itself" — there is no `solution/` wrapper.

```
case/
  constant/  system/  reports/        # the usual case payload
  FERMENTATION/  CONCENTRATION/  ...   # fractal SECTOR folders (alphabetic)
  0/                                   # instant 0 (the initial seed)
    streams                              # ROOT flat stream table (the spine)
    FERMENTATION/ streams                # per-sector view
    CONCENTRATION/ streams
    byUnit/                              # per-unit breakout
  1/  2/  3/                           # later recycle iterations
  latest -> 3                          # convenience symlink to the highest
  solution.log                         # the instant index
```

Key idea. **A stream IS a surface field (ϕ); the OpenFOAM analogy is exact.** The payload file is named `streams`; `latest` is the highest-numbered instant (latestTime style — restart discovers it by the max integer dir name). A load-bearing *all-digit filter* on the numeric-dir scan guarantees a sector folder (alphabetic) is never swept or mistaken for an instant, and the fractal loader iterating declared children (not a filesystem walk) guarantees the reverse.

The pilot is `tutorials/plant/ChemicalPlantTutorial`; `writeInterval` controls how many instants land (set it to `1` on a 3-iteration case or you only see `0` and the final — and miss the whole tear-residual march).

19 The data cascade — arity, scope, kind

> *What you'll learn.* The mechanical decision procedure for WHERE every thermodynamic number lives. This is the engine-side summary; the canonical, ratified reference is `docs/ai/data-doctrine.md` — read it before any curation act.

Key idea. A datum's home is decided by ARITY, then SCOPE, then KIND — in that order. Read the quantity's *definition*, not its frequency of use.

(1) Arity — how many species does the definition name?

- **Arity-1** (the molecule alone on the table, no partner named — *not even implicitly*) → INTRINSIC, `data/standards/components/<n` T_c, P_c, ω, MW, `gibbsFormation`, `solid`, `liquidPure`, `transport`.
- **Arity-2** (the definition must name a partner — a solvent, a counter-ion, an interaction species, *even water*) → PAIR/SET data in a **catalogue**, keyed by the pair, cited per value, referenced by name. NRTL τ, Henry H, Pitzer β, EoS k_{ij}, a heat of solution. **Never copied into the `.dat`.**

(2) Scope — at what level is the number TRUE? A datum lives at the *highest level where it is true* (the canonical value is a curation act in `data/standards/`). A lower node overlays only when it makes the value *more* true — a sample-measured refinement. Precedence, low to high: `proposed < standard < case < sector < unit`.

(3) Kind — the molecule, or the molecule-in-a-machine? A *rate* (kinetics) or a *geometry / distribution* (PSD) is NOT component data at all — it lives in the operation's `constant/` (`constant/crystallisation`, `constant/dryingKinetics`, `constant/reactions`). PSD is a *stream* attribute.

The aqueous TIER and the default-solvent resolver. Water gets *no* exemption from the arity test: an "in-water" property is pair data. Its one earned privilege is a single canonical, by-name aqueous reference tier — `data/standards/electrolyte/ions.dat` for ions, and `data/standards/solution/<solute>-<solvent>.dat` for molecular solutes (the heat of solution). The package declares a default solvent once (`solvent water;`); when an in-solvent property is needed and no solvent is named at the call site, the resolver uses the default **and announces it every run**:

```
[thermo] dHsoln(sucrose): solvent not named -> DEFAULT = water;
         solution/sucrose-water.dat  [Putnam & Kilday 1986]
```

Off-default it *fails with a remedy* ("provide solution/sucrose-ethanol.dat or set `solvent water;` explicitly") — it never silently substitutes the water number. This is implemented in `ThermoPackage` (`solventName_`, the `SolutionRegistry` walk).

The overlay merges block-by-block. As in §10: a reference-state block (`solid{}`, `gasIdeal{}`, `liquidPure{}`, each `eosParameters.<model>{}`) is the atomic unit of physical meaning. An overlay replaces the whole block — the curator copies the whole block they refine; a lone-scalar overlay is the forbidden hidden hybrid.

Model extensibility — `eosParameters{ <model>{...} }`. The same definition test, asked of the *model*:

1. A model needing only the corresponding-states triad $\{T_c, P_c, \omega\}$ (every cubic: SRK, PR, ...) **reads the triad off the Component and derives its own $a_c, b, m, \alpha(T)$ in source** — adding it touches *zero* data files (this is why adding PR after SRK touched no `.dat`). Do not pre-bake a_c into the `.dat`.
2. A model-specific PURE parameter that cannot be generated (PC-SAFT's $m, \sigma, \varepsilon/k$) goes in a `model`-keyed sub-block on the component, `eosParameters{ PCSAFT{...} }`, each factory reading *its own* sub-block and ignoring the rest.
3. PAIR parameters (k_{ij}) go in a model-keyed catalogue, `data/standards/eos/<model>/binaryInteractions/<i>-<j>.dat`, with the always-permitted inline override in the package.

A model with no required `eosParameters.<model>` block **fails with a remedy**, never a silent corresponding-states fallback.

20 The elements datum — one enthalpy surface

> *What you'll learn.* Why the energy balance runs on ONE datum, how the heat of reaction / solution / crystallisation emerge from it, and what throws when a datum is missing.

A heat balance that has to close *through* a reactor, a crystalliser, or a dissolution cannot use a sensible (arbitrary-T_{ref}) datum — it would silently drop the heat of reaction. Choupo runs the energy ledger on the **elements datum**: formation enthalpies at 298.15 K, the same reference as the ideal-gas H_{ig}, so the reaction / phase-change / dissolution enthalpy *emerges* as the difference of formation enthalpies.

> **Key idea. One datum, no silent fallback (Vítor's law: elements-only, throw-on-gap).** The second datum and every silent path were deleted — `streamH_sensible`, `solidH_sensible`, the `tryDatum(elements = false)` fallback, the `EnergyRef::Sensible` arm. `streamH_elements` now *propagates a real missing-Hf gap*, naming the component, instead of swallowing it. A curation gap is loud, never a quietly wrong closure.

The solid term. A stream's enthalpy is computed over both phases: the fluid (`s.F`, `s.z`) **and** the solid carried in `s.s[]` (kmol/s per component), on the same elements datum — $\sum_i s.s[i] \cdot h_i^\circ(\text{solid}, T)$, the same solid formation leg `Component::h_formation(T,"solid")` uses. Before this, the crystals a crystalliser had just made were silently dropped from the stream enthalpy and the heat of crystallisation read zero; counting `s.s[]` (plus the aqueous solution tier for the dissolved solute) closed the crystalliser balance.

Utility legs vs process streams — no double-count. The ledger partitions each unit's ins/outs by `category` (`src/reporting/BalanceMath.H`, `EnergyBalanceReport.cpp`): utility-tagged legs (steam → condensate) feed a `qBoundary` accumulator — the real heat crossing the boundary — while process streams feed $h_{\text{in}}/h_{\text{out}}$. An evaporator's `duty_kW` is delivered *through its own steam stream*, so it is INTERNAL and must NOT also be summed as an external item; a process-to-process exchanger's duty is likewise internal. Closure $= 100 \cdot \Delta H/$supplied over genuine boundary heat only. This killed a fake 2131 % non-closure that was a pure double-count, not lost latent heat.

One enthalpy surface everywhere. The published stream `.H` (Flowsheet) and the isothermal-flash duty (`IsothermalFlash.cpp`) now run on the *same* canonical elements-datum form the report reads (`H_stream_formation`, $h_{ig} - \Delta h_{\text{vap}}(T)$), not a Watson-at-298 + $\int c_p^{\text{liq}}$ variant. The two differed by the liquid latent model and surfaced as a $\sim 24\,\%$ gap between a flash's reported Q and its own stream ΔH; unifying the surface moves only the reported *duty* (an energy KPI), never the material flows.

21 Extending Choupo — step by step

> *What you'll learn.* Two complete, copy-pasteable walkthroughs — *tim-tim-por-tim-tim* — each ending in a green `bin/runTests`. Recipe A: a new unit operation, end to end. Recipe B: a new property-prediction method, with the validation-first step that earns your trust before you simulate with it.

21.1 Recipe A — a new unit operation, end to end

The worked example: `isothermalSplitter` — a unit that splits one feed into two outlets by a fraction θ to the top, both outlets at the feed's T, P, z (composition unchanged; the spec is the split fraction). Trivially simple physics on purpose — so every *mechanical* step is visible.

Step 1 — The header. `src/unitOperations/mixer/IsothermalSplitter.H` (it belongs beside the existing `Splitter`):

```
#pragma once
#include "core/Dictionary.H"
#include "thermo/ThermoPackage.H"
#include "unitOperations/UnitOperation.H"

namespace Choupo {

class IsothermalSplitter : public UnitOperation {
public:
    IsothermalSplitter() = default;
    const std::string& type() const override { return type_; }

    int solve(const DictPtr& dict,
              const ThermoPackage& thermo,
              int verbosity) override;

    std::vector<ProcessStream>      producedStreams() const override { return produced_; }
    std::map<std::string, scalar>   kpis()            const override { return kpis_; }

private:
    inline static const std::string type_ = "isothermalSplitter";
    std::vector<ProcessStream>      produced_;
    std::map<std::string, scalar>   kpis_;
};

} // namespace Choupo
```

Note the exact `solve` signature — `int solve(const DictPtr&, const ThermoPackage&, int)` — and that `type()` returns a reference to a member string. Prepend the licence banner (§on header banner); it doubles as the licence notice.

Step 2 — The implementation. `src/unitOperations/mixer/IsothermalSplitter.cpp`. The Flowsheet injects the inlet and your parameters as sub-dicts of `dict`; you publish outlets into `produced_` in the order your `outputs (... )` declares:

```
#include "mixer/IsothermalSplitter.H"
#include "core/Dimensions.H"
#include <iostream>

namespace Choupo {

int IsothermalSplitter::solve(const DictPtr& dict,
                              const ThermoPackage& thermo,
                              int verbosity)
{
    auto feed = dict->subDict("feed");          // F (kmol/s SI), Tfeed, Pfeed
```

```
    auto comp = dict->subDict("composition");   // mole fractions z_i
    auto op   = dict->subDict("operation");     // YOUR parameters

    const scalar F     = feed->lookupScalar("F",     Dims::molarFlow);
    const scalar Tfeed = feed->lookupScalar("Tfeed", Dims::temperature);
    const scalar Pfeed = feed->lookupScalar("Pfeed", Dims::pressure);
    const scalar theta = op->lookupScalar("splitFraction");    // [-], 0..1

    if (theta < 0.0 || theta > 1.0)
        throw std::runtime_error(
            "isothermalSplitter: splitFraction must be in [0,1]");

    std::vector<scalar> z(thermo.n());
    for (std::size_t i = 0; i < thermo.n(); ++i)
        z[i] = comp->lookupScalar(thermo.comp(i).name());

    if (verbosity >= 3)
        std::cout << "  [isothermalSplitter] F = " << (F * 3600.0)
                  << " kmol/h, split " << theta << " to top\n";

    produced_.clear();                          // ORDER == outputs(...) order
    ProcessStream top;  top.name = "top";
    top.F = theta * F;  top.T = Tfeed;  top.P = Pfeed;  top.z = z;
    produced_.push_back(top);

    ProcessStream bot;  bot.name = "bottom";
    bot.F = (1.0 - theta) * F;  bot.T = Tfeed;  bot.P = Pfeed;  bot.z = z;
    produced_.push_back(bot);

    kpis_.clear();                              // KPIs feed sizing/costing/sweep
    kpis_["F_top"]         = top.F;
    kpis_["F_bottom"]      = bot.F;
    kpis_["splitFraction"] = theta;

    return 0;                                   // 0 == converged
}

} // namespace Choupo
```

Key idea. **Fill EVERY field of a produced stream** (`F`, `T`, `P`, `z`, and `vf` / `s` if relevant). A half-filled stream propagates NaN downstream and the NaN guard fails the case — by design.

Step 3 — Register it (the one line). Edit `src/unitOperations/UnitOperation.cpp`: add the include beside the other mixer includes, and one line in `registerBuiltins()`:

```
#include "mixer/IsothermalSplitter.H"       // (1) beside #include "mixer/Splitter.H"
//...
void UnitOperation::registerBuiltins()
{
    auto reg = [](const std::string& name, auto&& maker) {
        registerType(name, std::forward<decltype(maker)>(maker)); };
    //...
    reg("splitter",           []{ return std::make_unique<Splitter>(); });
    reg("isothermalSplitter", []{ return std::make_unique<IsothermalSplitter>(); }); // (2)
    //...
}
```

That is the *entire* registration story — no macro, no static initialiser. A student greps `registerBuiltins` and sees your type listed.

Step 4 — Build (the recursive pickup).

```
make all
```

The Makefile discovers sources with `find src -name '*.cpp'`, so the new `.cpp` is compiled with no Makefile edit; `-MMD -MP` tracks the new header dependency automatically.

Step 5 — Rebuild the WASM (MANDATORY).

```
make wasm-gui          # = choupoSolve + choupoProps, the two the GUI uses
```

> **Common pitfall.** Skip this and the GUI throws `UnitOperation::New: unknown type 'isothermalSplitter'` even though the native binary runs the type fine. The WASM solver is a separate Emscripten compile; a new built-in only reaches the browser after a WASM rebuild. If `make wasm` reports "Nothing to be done", force it: `make wasm-clean && make wasm`. Never run two `make wasm` concurrently — they clobber `gui/public/wasm/`.

Step 6 — A minimal tutorial case. Keep it trivial so it doubles as a regression test. `tutorials/steady/mixer/splitter01`

```
splitter01_isothermal/
  splitter01.cho                # empty marker (the GUI's openable entity)
  system/
    controlDict                 # application choupoSolve; verbosity 3;
    flowsheetDict               # the topology below
  constant/
    thermoPackage               # components ( benzene toluene ); ideal
```

The `system/flowsheetDict` (one feed boundary, one unit, two product boundaries):

```
boundary
{
    inlets  ( feed );
    feed { F 1.0 kmol/s; Tfeed 350 K; Pfeed 1 bar;
            composition { benzene 0.5; toluene 0.5; } }
}

units
(
    {
        name      S1;
        type      isothermalSplitter;
        in        feed;
        outputs   ( top bottom );        // ORDER matches produced_ order
        operation { splitFraction 0.3; }
    }
);
```

Step 7 — Add KPIs to the regression (`expected`). Your `kpis()` already feed sizing, costing and the sweep/optimisation drivers; pin the load-bearing ones to golden masters. Drop a `tutorials/steady/mixer/splitter01_isothermal/e`

```
# kind   name   key            expected   reltol
kpi      S1     F_top          0.3        1e-6
kpi      S1     F_bottom       0.7        1e-6
stream   top    F              0.3        1e-6
```

then record/verify and run the full gate:

```
bin/runTests --record tutorials/steady/mixer/splitter01_isothermal   # write from a trusted run
bin/runTests                                                          # the whole corpus must
    ↪ stay green
```

> **Key idea.** **The gate is `bin/runTests`, not a bare exit-code loop** — NaN/inf guard on *every* case plus the `expected` KPI/stream comparison (§14). A solver can return NaN with exit code 0; only the structured-result scan catches it.

21.2 Recipe B — a new property-prediction method

A property model plugs into one of four bases. **Pick the base by the quantity you predict**, then follow the same `fromDict` + `registerModel` pattern — and validate against measured data *before* you trust it.

You predict...	Base	`registerModel` factory signature
$P^{\mathrm{sat}}(T)$	`VaporPressureModel`	`(const DictPtr&)`
$c_p(T)$	`HeatCapacityModel`	`(const DictPtr&)`
$\gamma_i(T, x)$	`ActivityModel`	`(const DictPtr&, const std::vector<std::string>&)`
φ_i, Z, H^{res}	`EquationOfState`	`(const DictPtr&, const std::vector<Component>&)`

B.1 — A vapour-pressure model (the simplest base). Say you add a two-parameter `Clapeyron` P^{sat}.

1. `src/thermo/vaporPressure/Clapeyron.{H,cpp}`, inheriting `VaporPressureModel`, override the pure virtual `scalar Psat_Pa(scalar T_K) const`; a constructor `Clapeyron(const DictPtr&)` reads the parameters (mirror `Antoine.cpp`, whose ctor is `Antoine::Antoine(const DictPtr&)`).
2. Register — include + one line in `VaporPressureModel::registerBuiltins()`:

```
#include "Clapeyron.H"
//...
void VaporPressureModel::registerBuiltins()
{
    registerModel("Antoine",
        [](const DictPtr& d) -> std::unique_ptr<VaporPressureModel>
        { return std::make_unique<Antoine>(d); });
    registerModel("AmbroseWalton",
        [](const DictPtr& d) -> std::unique_ptr<VaporPressureModel>
        { return std::make_unique<AmbroseWalton>(d); });
    registerModel("Clapeyron",                                    // new
        [](const DictPtr& d) -> std::unique_ptr<VaporPressureModel>
        { return std::make_unique<Clapeyron>(d); });
}
```

3. Select it in a component's `vaporPressure{ model Clapeyron; ... }` block.

B.2 — Validate FIRST (the AAD step). A property model is **not trustworthy until it has been checked against measured data**. Choupo's `choupoProps` binary is the validation weapon: run the *measured* dataset through the model and read the average absolute deviation (AAD) before you let the model into a flowsheet.

```
choupoProps <validationCase>          # e.g. a propertyScan1D over measured T
# the run prints, per point:  model vs measured, dev %;  headline:
#   AAD = 1.8 %   AAD tolerance = 3.0 %   -> PASS
```

> **Key idea.** **The number that earns trust is the AAD against measured data, not that the model compiles.** The bench prints the per-point deviation and a headline AAD against a tolerance (the same machinery `HeatTransferBench` and the Pitzer activity validation use); a model that misses its anchor beyond tolerance fails loudly. This is the property half of "see, then decide" — you simulate with a model *because* you have seen its AAD, never because a dropdown recommended it.

B.3 — An equation of state (and the data-doctrine path). An EoS is the one base where WHERE the parameters live is a doctrine decision. The canonical reference is `docs/ai/data-doctrine.md` §4; in short:

1. **Corresponding-states EoS (reads only the triad).** If the model needs only $\{T_c, P_c, \omega\}$, **read them off the `Component` and derive the rest in source** — touch *zero* data files. This is exactly what `SRK` and `PR` do: the `SRK` constructor reads `c.Tc()`, `c.Pc()`, `c.omega()` and computes $a_c = \Omega_a R^2 T_c^2 / P_c$, $b = \Omega_b R T_c / P_c$, $m = m(\omega)$. Do *not* pre-bake a_c into the `.dat` (it would duplicate a value the model owns and let the two drift).
2. **Model-specific PURE parameters (cannot be generated).** PC-SAFT's $\{m, \sigma, \varepsilon/k\}$ live in a `model`-keyed sub-block on the component — additive, never disturbing the triad:

```
// inside the component .dat
eosParameters
{
    PCSAFT  { m 2.4653;  sigma 3.6478e-10;  epsilon_k 287.35;
              provenance { origin regressed; method "Gross & Sadowski 2001"; } }
    // SRK/PR appear in NO key here -- they read Tc, Pc, omega
}
```

   Each factory reads *its own* `eosParameters.<model>` sub-block by name and ignores the rest, so one molecule carries SRK + PR + PC-SAFT simultaneously, each with its own `provenance{}`.
3. **Pair parameters k_{ij}.** A model-keyed catalogue, `data/standards/eos/<model>/binaryInteractions/<i>-<j>.dat`, with the always-permitted inline override. `SRK::fromDict` already reads an inline `binaryInteractions` list of `{ i; j; kij; }` dicts and resolves names positionally against the component list.

The explicit factory, and the open follow-up. Register the EoS the same explicit way, dispatching to a `fromDict` static (the `Component`-list overload):

```
registerModel("PCSAFT",
    [](const DictPtr& dict, const std::vector<Component>& comps)
        -> std::unique_ptr<EquationOfState>
    { return PCSAFT::fromDict(dict, comps); });
```

> **Common pitfall.** **Glass-box requirement:** an EoS with no required `eosParameters.<model>` block on a component must **fail with a remedy** ("PCSAFT needs m, sigma, epsilon_k for X; fit them with choupoProps or pick SRK/PR which run off Tc, Pc, omega"), *never* a silent corresponding-states fallback. **Open follow-up** (noted in the doctrine): a uniform `modelParams` accessor on `Component` that hands a factory its `eosParameters.<model>` sub-dict by model name is not yet generalised — today each EoS that needs a pure block reads it itself. Until that lands, copy the `liquidViscosity{}` raw-sub-dict reader pattern.

22 Roadmap

Ordered by impact for the MCFT course and Vítor's research:

Target	Idea	Difficulty
(shipped)	Levenberg-Marquardt parameter-estimation driver (`fitBinaryPair`)	—
	Nelder-Mead optimisation driver (minimise cost / TAC)	easy
	LL flash via 2nd-order Michelsen TPD with multiple restarts	hard
	Solids: inert phase + filter / cyclone / dryer unit-ops	medium
	Crystalliser + PSD basics	hard
	NRTL distillation robustness (Naphtali-Sandholm)	medium-hard
v1.0	Equation-oriented sparse Newton for the full flowsheet	very hard
> v1	Membrane NF/RO unit op (Vítor's research bridge)	research-grade

Tech-debt items also welcome:

- Reference logs for every tutorial → in-tree regression diff.
- Reaction-kinetics library (Arrhenius, Langmuir-Hinshelwood, Michaelis-Menten) as a `Kinetics` base class.

23 Things to NOT do

> *What you'll learn.* A short list, each item born from a real incident. Memorise it.

- Do **not** suggest a Python rewrite or wrapper. The course teaches numerical methods in C++; rewriting in Python would defeat the purpose.
- Do **not** add macro auto-registration. Explicit `registerBuiltins()` is non-negotiable (see §6).
- Do **not** add a heavy dependency — even a "small" one. Pedagogical visibility outweighs convenience.
- Do **not** split a binary by numerical strategy (`*Foam`-style proliferation). Strategy is run-time. Splitting by *problem class* — the four binaries Solve / Batch / Ctrl / Props — is the deliberate exception, not a precedent for more.
- Do **not** bypass the dict format. Never use YAML, JSON, or TOML inside the case directory. The dict is the canonical wire format.
- Do **not** relitigate decisions in `CLAUDE.md` §10 without first reading what's already been discussed and why.
- Do **not** merge code that breaks the regression, even if the failure is "obviously unrelated".

24 Quick reference

Task	Command / file
Build release	`make all`
Build debug	`make MODE=debug all`
Clean	`make clean` / `make distclean`
Source env	`source etc/bashrc`
Run a case	`runCase tutorials/<name>`
Run from inside case dir	`runCase`
List tutorials	`listCases`
List built-in unit-op types (in code)	`UnitOperation::availableTypes()`
Tweak a dict field programmatically	`dict->setScalarAtPath(path, v)`
Add a unit op (full walkthrough)	§21.1 (quick: §7)
Add a property model (full walkthrough)	§21.2 (quick: §8)
Add a component / material / reaction	§10
Add an outer driver / post-processor	— (see the outer-driver section)
Where does a number live?	§19 + `docs/ai/data-doctrine.md`
Energy datum / balance closure	§20
Fractal flowsheet + instants	§17, §18
Project conventions / AI rules	`CLAUDE.md`
User-facing how-to	`docs/userGuide.pdf`

Colophon. This manual was typeset in Latin Modern with the Choupo shared LaTeX preamble. Sources live in `docs/`; rebuild with `cd docs && make`.